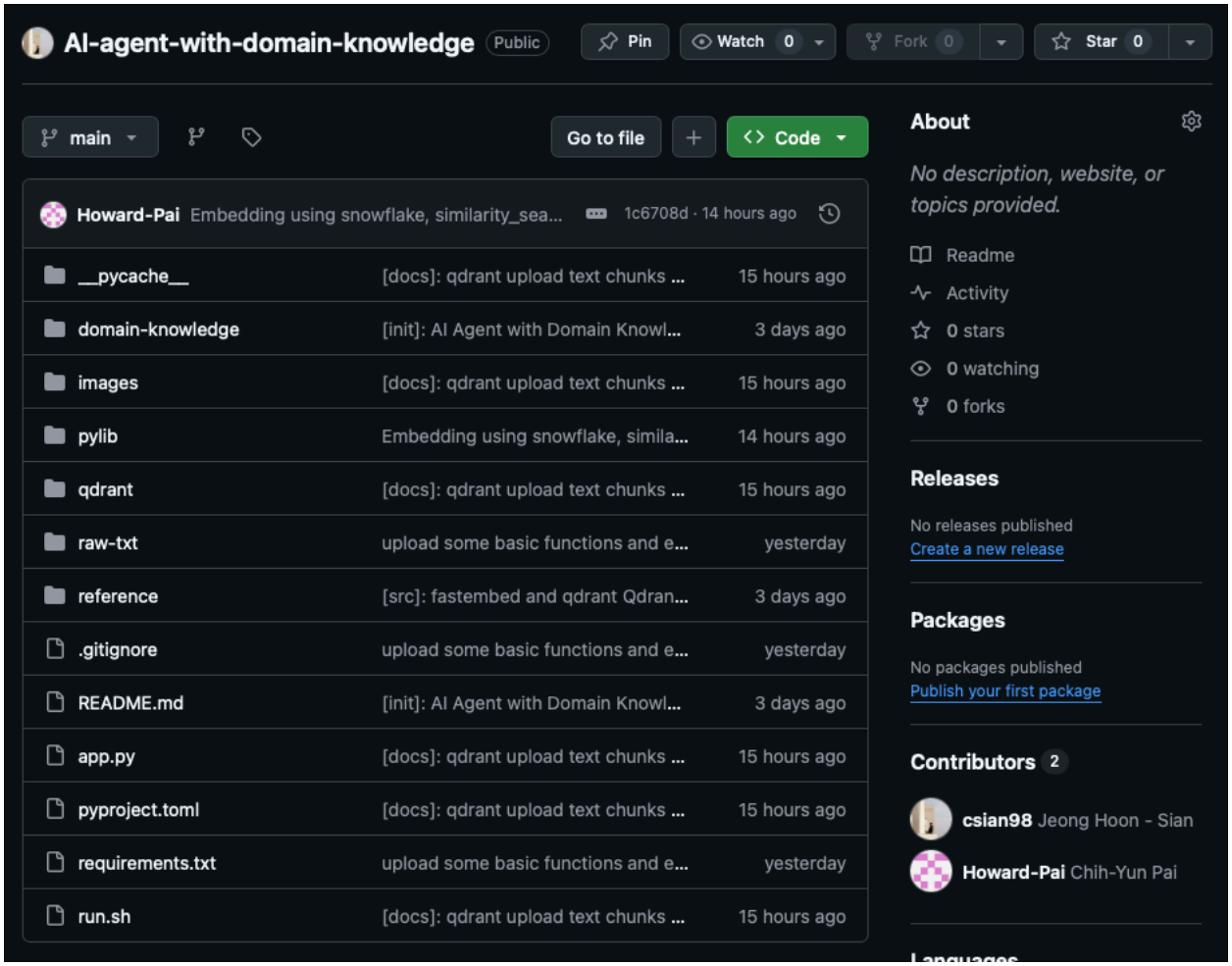
DSCI 560
Data Science Professional Practicum

Laboratory Assignment 9

Group 15: Jeong Hoon Choi, Viraj Bansal, Chih-Yun Pai



Git Hub Repository: https://github.com/csian98/AI-agent-with-domain-knowledge

Data Collection / Processing /Storage & Vector Embedding

For data collection, we used two PDF documents as our knowledge base: *Ads Cookbook* and *Licensing*. These were sourced locally and processed using a custom Python extraction pipeline built on `pdfplumber`. Other pdf documents can be uploaded later on.

For processing, we developed a column-aware PDF text extractor that detects whether a page uses a two-column layout by checking if more than 15% of words start past the page midpoint. If so, the page is cropped and each column is extracted independently to preserve reading order. This avoids the garbled output that results from naively extracting text line-by-line across both columns. Page markers are then stripped, and the cleaned text is split into overlapping chunks of 500 characters with a 50-character overlap using LangChain's `CharacterTextSplitter`. Chunking is necessary because embedding models and LLMs have input size limits, and smaller semantically coherent chunks also improve retrieval precision.

```python
def _is_two_column(page: page) -> tuple[bool, float]:
    """Return (is_two_col, split_x) for the page."""
    words = page.extract_words() # words = a list of dictionaries, containing att
    if not words:
        return False, None

    mid = page.width * COLUMN_SPLIT_RATIO
    # Allow a 40-pt dead-zone around the midpoint to avoid false positives
    left  = [w for w in words if w["x1"] < mid - 20]
    right = [w for w in words if w["x0"] > mid + 20]

    if not right:
        return False, None

    right_fraction = len(right) / len(words)
    if right_fraction < TWO_COL_THRESHOLD:
        return False, None

    # Estimate the actual split by finding the most common left-edge of
    # right-column words (rounded to the nearest 10 pts).
    right_x0s = [w["x0"] for w in right]
    split_x = min(right_x0s) - 10        # a little padding
    return True, split_x
```

For vector embedding, we used Snowflake's EMBED_TEXT_1024 function with the snowflake-arctic-embed-l-v2.0 model, which produces 1024-dimensional dense vector representations of each chunk. Chunks are embedded in batches of 20 using a single SQL round-trip per batch via UNION ALL to stay within Cortex rate limits while minimizing latency.

```
-> 111 chunks
Embedding 1–20/111...
Embedding 21–40/111...
Embedding 41–60/111...
Embedding 61–80/111...
Embedding 81–100/111...
Embedding 101–111/111...
Done — 111 chunks stored for 'licensing'.
```

For storage, embeddings are stored in a Snowflake table with a VECTOR(FLOAT, 1024) column alongside the source name, chunk ID, and raw content. Snowflake was chosen because it natively supports vector similarity search via VECTOR_COSINE_SIMILARITY, eliminating the need for a separate vector database and keeping the entire pipeline within one platform.

	ID	SOURCE	··· CHUNK_ID	CONTENT	EMBEDDING
1	1931	lab1-s26	0	Data Science Profession Laboratory Instructor: Yo NOTE: You must use GitHub to...	[-0.029226,-0.028511,0.0
2	1932	lab1-s26	1	nal Practicum (DSCI 560) y Assignment 1 oung Cho, Ph.D. urce code and reports...	[-0.004677,-0.008072,0.0
3	1933	lab1-s26	2	DSCI-560 ● When prompted, choose the Ub installation medium. ● The Ubuntu...	[-0.040165,-0.016302,0.0

Vector Embedding & Long-Term Memory-Referencing LLM

Additionally, our team generated embeddings for the raw chunks stored in Snowflake using Fastembed's BAAI/bge-small-en-v1.5 model, and subsequently stored them locally using Qdrant. (Save in the same format as the Snowflake Embedding above)

```
[kyra@giovanni lab9]$ uv run pylib/qdrant_upload_main.py
Fetching 5 files: 100%|                                                    | 5/5 [00:06<00:00,  1.31s/it]
Download complete: : 67.2MB [00:06, 12.1MB/s]          /home/kyra/lab9/.venv/lib/python3.9/site-packages/boto3/compa
t.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting April 29, 2026. To continue receivin
g service updates, bug fixes, and security updates please upgrade to Python 3.10 or later. More information can be found
 here: https://aws.amazon.com/blogs/developer/python-support-policy-updates-for-aws-sdks-and-tools/
  warnings.warn(warning, PythonDeprecationWarning)
Download complete: : 67.2MB [00:08, 7.62MB/s]
[353/353] Insert into qdrant...
Complete uploading...
```

Quadrant provides the infrastructure to perform large-scale vector searches quickly and reliably within LLM Agent-RAG systems and provide Vector+Metadata filtering. Internally within LLM Agent-RAG systems. Internally, it utilizes Approximate Nearest Neighbor (ANN) search likes Hierarchical Navigable Small World (HNSW); unlike traditional algorithms–which incur an O(N) time complexity when searching for cosine similarity–this approach allows for significantly faster search speeds with a time complexity of just O(logN).

We offer the use of three LLM models, including the two Vector Embedding Databases mentioned above. Internally, we can utilize 'gpt-oss:20b' running locally via the Ollama API, and we additionally provide OpenAI's 'gpt-5-nano' and Anthropic's 'claude-haiku-4-5', both of which retrieve and process results via APIs from external cloud services.

```
ollama_llm_engine = OllamaLLMEngine(model="gpt-oss:20b", stream=False)
openai_llm_engine = OpenAILLMEngine(model="gpt-5-nano", stream=False)
anthropic_llm_engine = AnthropicLLMEngine(model="claude-haiku-4-5", stream=False)
```

For the local model, we increased the default input context size from 4k to 16k to facilitate the injection of extensive domain knowledge, and we instructed the model to generate appropriate responses via the System prompt, as shown below. (gpt-oss:20b was trained on up to 128k input contexts).

```
PARAMETER temperature 1
PARAMETER num_ctx 16384

SYSTEM """
You are an AI Agent equipped with domain specific knowledge.
When provided with domain knowledge relevant to a user's query, you must prioritize
the use of that knowledge to formulate an appropriate response.
"""
```

The implemented flow was built using Python's FastAPI to enable it to be called from the web server's backend. To execute the local models, a personal computer equipped with a GPU was utilized; furthermore, OpenVPN was employed to establish a secure internal network connection between this machine and the web server.

```
[kyra@giovanni lab9]$ ./run.sh
INFO:     Will watch for changes in these directories: ['/home/kyra/lab9']
INFO:     Uvicorn running on http://0.0.0.0:4444 (Press CTRL+C to quit)
INFO:     Started reloader process [5191] using StatReload
INFO:     Started server process [5193]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Once the API has been exposed to external hosts (OpenVPN), the instructions for using 'curl' to access it are as follows.

```
~ » curl -X POST http://10.10.50.5:4444/api/generate -H "Content-Type: application/json" -d '{"model": "gpt-oss:20b", "e
mbed": "qdrant", "prompt": "How to make new workspaces in ADS2011?"}'
{"response":"## Quick-Start: Creating a New Workspace in **Keysight ADS 2011**\n\nA **workspace** is a self-contained fo
lder that contains the *lib.defs* file and all the libraries you want to work with.  \nOnce a workspace is created you c
an open it, add or copy cells, and start a new schematic or layout.\n\n> **Tip:** Keep the workspace name *and the path*
 free of spaces – ADS 2011 chokes on spaces in the path.\n\n---\n\n### 1. Launch ADS 2011\n\nOpen the ADS main window (n
ormally called **ADS 2011 – Main Window**).  \nYou should see the familiar toolbar, the Project Browser and the Design K
it.\n\n---\n\n### 2. Start the "New Workspace" Wizard\n\n1. **File** ► **New** ► **Workspace…**  \n   (You can also use
the "New Workspace" icon in the toolbar if it's shown.)\n\n---\n\n### 3. Set the Workspace Name & Location\n\n| Step | W
hat to do | Why |\n|------|------------|-----|\n| **Workspace name** | Type a concise name (e.g. `Lab19`) | Used in the
`lib.defs` path and in the ADS window title. |\n| **Workspace folder** | Choose a folder **without spaces** (e.g. `C:\\A
DSWorkspaces\\Lab19`) | ADS 2011 does not support spaces in paths. |\n| **Click **Next**** | | |\n\n> *The workspace fol
der will be created automatically.*\n\n---\n\n### 4. Select Libraries to Include\n\n| Option | Action | Notes |\n|------
--|--------|-------|\n| **Default libraries** | Keep the two default libraries (Analog/RF and DSP) checked | These are a
lways useful. |\n| **Add User Favorite Library/PDK** | Click the link to browse and add your own component libraries (e.
```

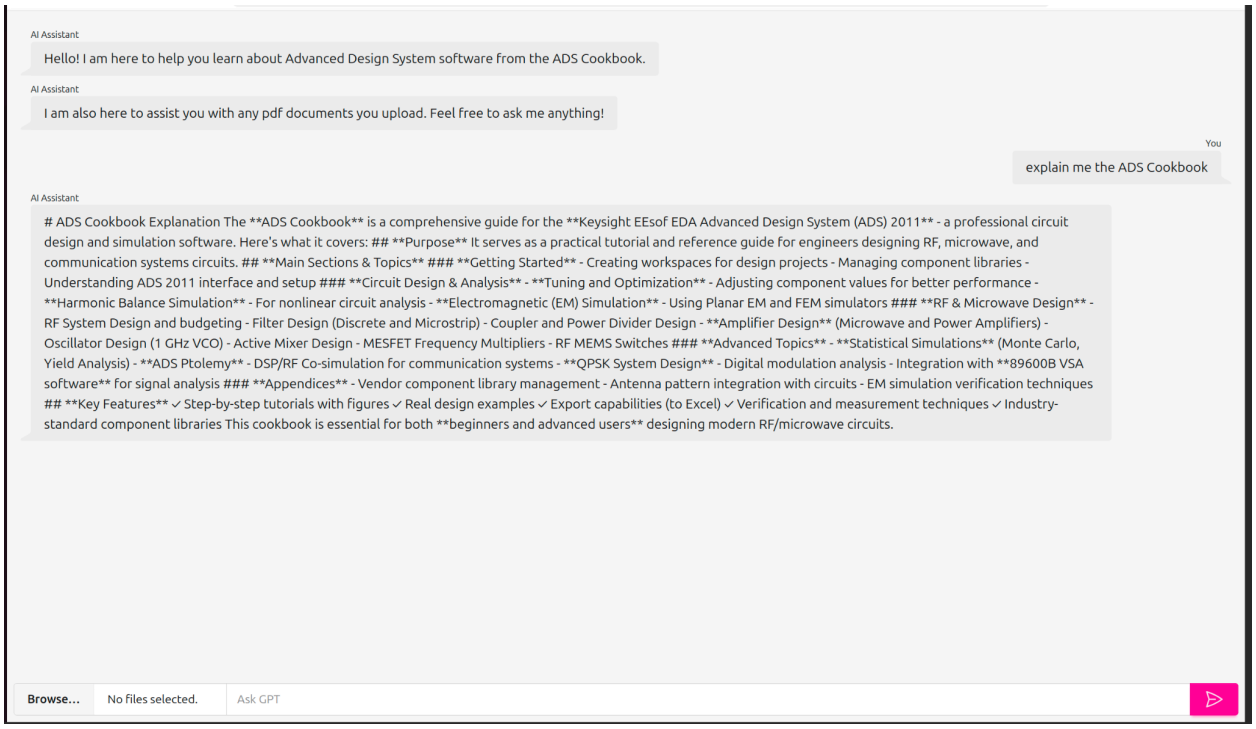
```
~ » curl -X POST http://10.10.50.5:4444/api/generate -H "Content-Type: application/json" -d '{"model": "gpt-5-nano", "em
bed": "snowflake", "prompt": "How to make new workspaces in ADS2011?"}'
{"response":"Here's how to create a new workspace in ADS2011:\n\n- Open ADS2011. In the main window, go to: File > New >
 Workspace.\n- Enter the workspace name. Note: the workspace name and the path to the workspace location must not contai
n spaces. Click Next.\n- Select the libraries to include in the workspace. ADS ships with the Analog/RF and DSP librarie
s by default. You can also add vendor/design libraries later (see "Add User Favorite Library/PDK" and unzipped libraries
 under oalibs/componentLib).\n- Provide a library name to organize designs within this workspace. This is a per-technolo
gy grouping (e.g., GaAs, GaN, InP, SiGe). This is not the same as vendor libraries.\n- Choose the preferred units for th
e design (e.g., mil) and set a layout resolution (e.g., 0.0001 mil). Click Next.\n- Finish the setup (Finish). A blank w
orkspace will open.\n\nTips:\n- If you need to use vendor libraries, unzip them first via DesignKit → Unzip Design Kit…,
 placing them in a central folder, then add them via the library management options.\n- You can create your first schema
tic after the workspace is created: File > New > Schematic.\n- You can copy existing cells between workspaces later (Cop
y Cell) or operate in cosimulation mode as described in the cookbook."}
```

```
~/Desktop » curl -X POST http://127.0.0.1:4444/api/upload -F "files=@lab1-s26.pdf" -F "files=@lab2-s26.pdf" -F "files=@l
ab3-s26.pdf"
{"status":"success"}%
```

An upload feature has been added to the API. This functionality allows the frontend to update the Snowflake and Qdrant databases with newly added PDF files, which can then be referenced to generate responses to user queries.

Web Interface Design and Implementation

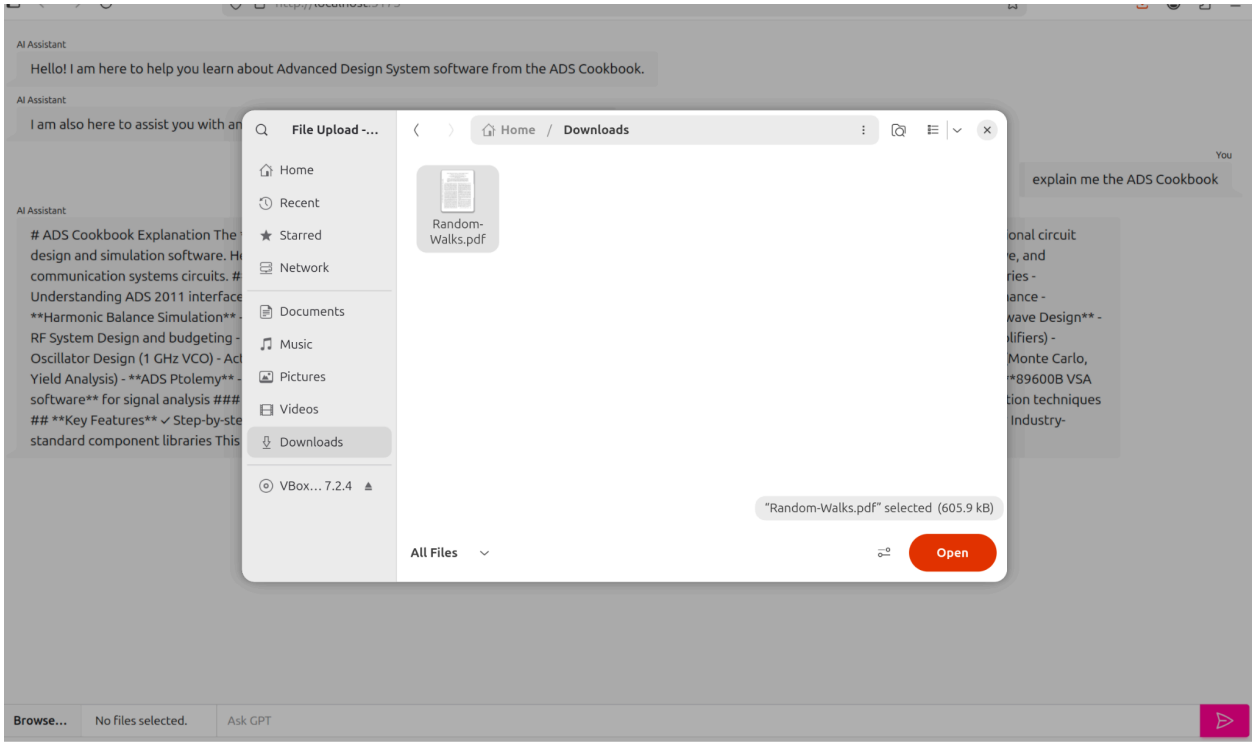
As with our previous frontend implementations, we decided to use Vite+React for building out the Chatbot web interface.



For styling the website, I used tailwind css which makes implementing styles with inline class names much easier than using raw css. We also used DaisyUI which is a React Component library that provides prebuilt components for us to simply customize and use within our React project instead of creating them from scratch. This allows us to ship more complex functionality faster. As you can see, we used various components from that library which includes input components such as textarea box, button, and file uploader as well as output components such as the chat bubbles you see displayed.

We used React Query for managing our api requests as they come with many built-in features such as automatic retries in case an api request times out.

The logic for this app is pretty simple in that messages passed by the user are relayed over an API call to our FastApi backend. This is also the case with the file uploader which also relays the uploaded pdfs in the API call for preprocessing, chunking, embedding and storing the pdf in Snowflake and QDrant vector databases. See picture below.



We also added some edge case logic such as allowing users to shift enter in the textarea box for separating out their query as well as disabling the textarea box during AI generation by the various chatbots in the backend.